ESP32-S3-LCD-1.28

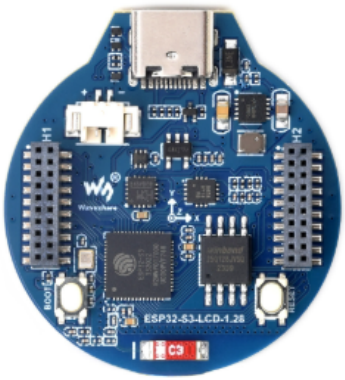
From Waveshare Wiki
Jump to: navigation, search

Overview

[Expand]

Version Description

This development board comes in both touch and non-touch versions. Please click on the corresponding product to view the detailed instructions.



(/wiki/ESP32-S3-LCD-1.28)

(/wiki/ESP32-S3-Touch-LCD-1.28)

ESP32-S3-LCD-1.28

(https://www.waveshare.com/esp32-s3-lcd-1.28.htm)

Type C USB, ESP32-S3R2

ESP32-S3-LCD-1.28
Non Touch Version

ESP32-S3-Touch-LCD-1.28 (/wiki/ESP32-S3-Touch-LCD-1.28)
Touch Version

Usage Instructions

ESP32-S3-LCD-1.28 currently provides two development tools and frameworks, **Arduino IDE** and **MicroPython**, providing flexible development options, you can choose the right development tool according to your project needs and personal habits.

Development Tools

(/wiki/File:180px-Arduino-IDE-logo.jpg)

Arduino IDE

Arduino IDE is an open source electronic prototyping platform, convenient and flexible, easy to get started. After a simple learning, you can start to develop quickly. At the same time, Arduino has a large global user community, providing an abundance of open source code, project examples and tutorials, as well as rich library resources, encapsulating complex functions, allowing developers to quickly implement various functions.

(/wiki/File:180px-MicroPython-logo.jpg)

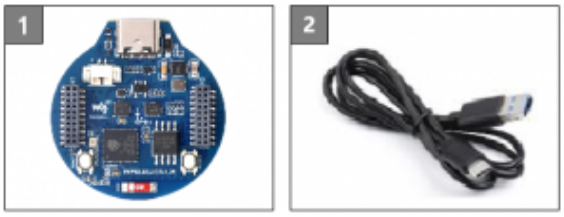
MicroPython

Micropython is a full implementation of the Python 3 programming language that runs directly on embedded hardware such as ESP32, Raspberry Pi Pico, etc. You can run Python scripts directly on the board through REPL, which is very suitable for rapid prototyping.

Arduino and MicroPython are suitable for beginners and non-professionals because they are easy to learn and quick to get started.

Components Preparation

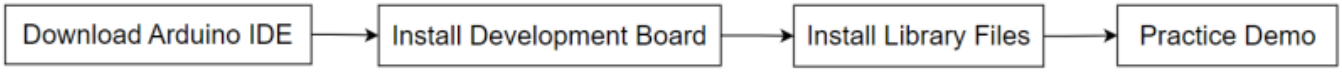
- ESP32-S3-LCD-1.28 x1
- USB cable (Type-A to Type-C) x 1



(/wiki/File:300px-ESP32-S3-LCD-1.28-uesnotes-01.png)

Working with Arduino

This chapter introduces setting up the Arduino environment, including the Arduino IDE, management of ESP32 boards, installation of related libraries, program compilation and downloading, as well as testing demos. It aims to help users master the development board and facilitate secondary development.

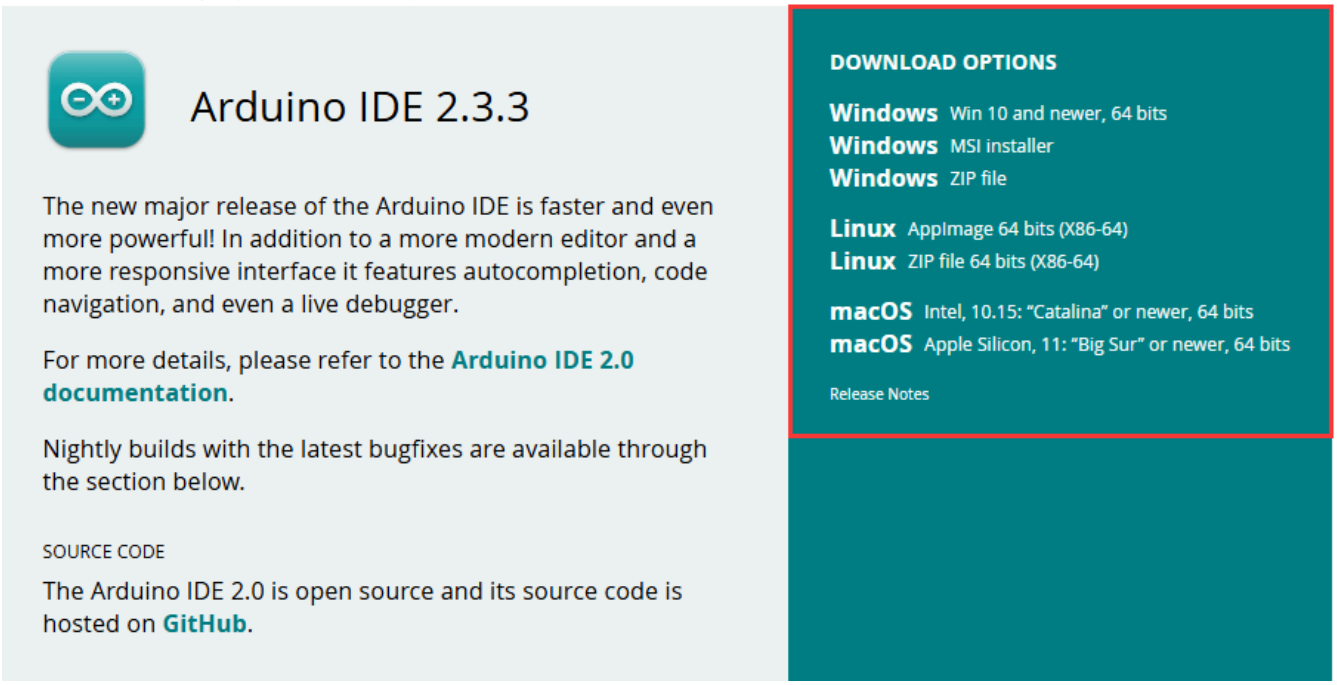


(/wiki/File:Arduino-flow-04.png)

Environment Setup

Download and install Arduino IDE

- Click to visit the Arduino official website (https://www.arduino.cc/en/software), select the corresponding system and system bit to download



(/wiki/File:ESP32-S3-AMOLED-1.91-Ar-software-01.png)

- Run the installer and install all by default

> The environment setup is carried out on the Windows 10 system, Linux and Mac users can access Arduino-esp32 environment setup (https://docs.espressif.com/projects/arduino-es

p32/en/latest/installing.html) for reference. (To use on Mac, you need to install the MAC driver (https://files.waveshare.com/wiki/common/CH34XSER_MAC.7z).)

Install ESP32 development board

- Regarding ESP32-related motherboards used with the Arduino IDE, the **esp32 by Espressif Systems** library must be installed first.
- According to **Board installation requirement**, it is generally recommended to use **Install Online**. If online installation fails, use **Install Offline**
- For the installation tutorial, please refer to Arduino board manager tutorial (https://www.waveshare.com/wiki/Arduino_Board_Managers_Tutorial)
- **esp32 by Espressif Systems** development board comes with an offline package. Click here to download: esp32_package_2.0.12_arduino offline package (https://drive.google.com/drive/folders/1Pcs_A4FKWvdSHnz9lEBYqOpr-noTMbIv?usp=sharing)

- ESP32-S3-LCD-1.28 required development board installation description

Board name	Board installation requirement	Version number requirement
esp32 by Espressif Systems	"Install Offline" / "Install Online"	2.0.12

Install library

- When installing Arduino libraries, there are usually two ways to choose from: **Install online** and **Install offline**. **If the library installation requires offline installation, you must use the provided library file**
  For most libraries, users can easily search and install them through the online library manager of the Arduino software. However, some open-source libraries or custom libraries are not synchronized to the Arduino Library Manager, so they cannot be acquired through online searches. In this case, users can only manually install these libraries offline.
- For library installation tutorial, please refer to Arduino library manager tutorial (https://www.waveshare.com/wiki/Arduino_Library_Manager_Tutorial)
- ESP32-S3-LCD-1.28 library file is stored in the sample program, click here to jump: ESP32-S3-LCD-1.28 Demo

- ESP32-S3-LCD-1.28 library installation description

Library Name	Description	Version	Library Installation Requirement
LVGL	Graphical library	v8.3.10	"Install Offline"
TFT_eSPI	LCD library	v2.5.34	"Install Offline"
TFT_eSPI_Setups	Custom library	--	"Install Offline"

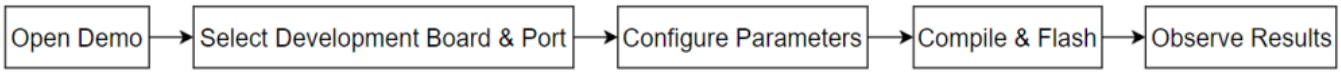
For more learning and use of LVGL, please refer to LVGL official documentation (https://docs.lvgl.io/master/intro/introduction/index.html)

Run the First Arduino Demo

If you are just getting started with ESP32 and Arduino, and you don't know how to create, compile, flash, and run Arduino ESP32 programs, then please expand and take a look. Hope it can help you!

[Expand]

Demo

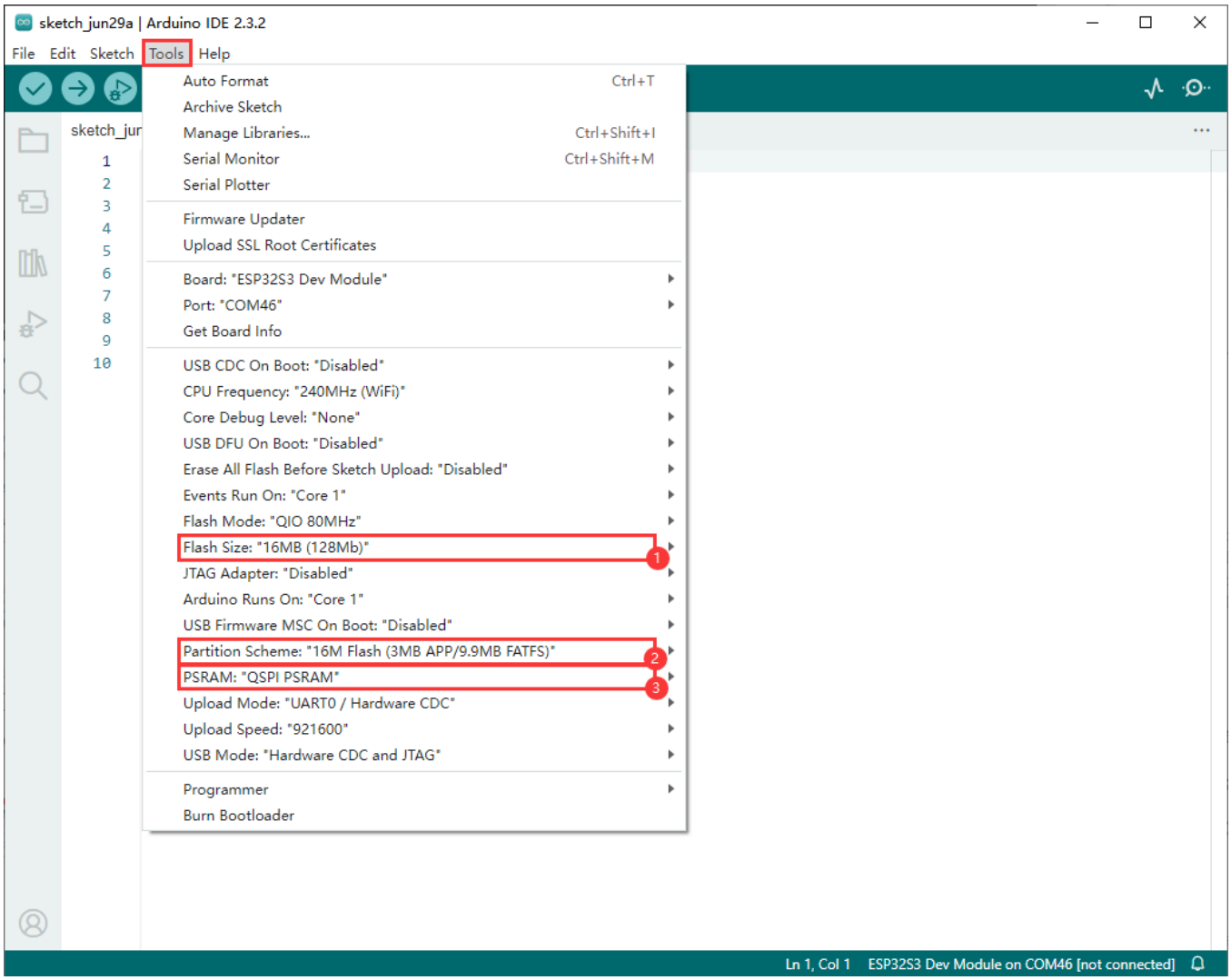


(/wiki/File:Demo-flow-01.png)

- ESP32-S3-LCD-1.28 demos

Demo	Basic Description	Dependency Library
ESP32-S3-LCD-1.28-Test	Test onboard device functionality	---
LVGL_Arduino	Display LVGL benchmark, music, etc	LVGL, TFT_eSPI , TFT_eSPI_Setups
LVGL_Chinese_Font	Display the 1000 built-in Chinese character fonts of LVGL	LVGL, TFT_eSPI, TFT_eSPI_Setups
LVGL_Chinese_7500_Char	Display the 7500 built-in Chinese character fonts of LVGL	LVGL, TFT_eSPI, TFT_eSPI_Setups

Arduino project parameter setting



(/wiki/File:ESP32-S3-Touch-LCD-1.28_Add_1.png)

ESP32-S3-LCD-1.28-Test

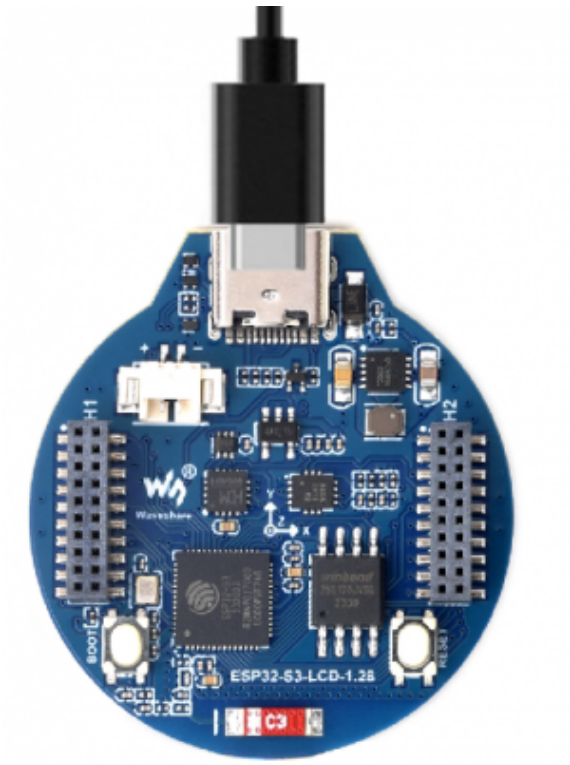
[Collapse]

Demo description

- This demo is used to test the use of screens, six-axis sensors, BAT

Hardware connection

- Connect the development board to the computer



(/wiki/File:300px-ESP32-S3-LCD-1.28-demo-01.png)

Code analysis

- **setup()**
  - Initialize the serial port and set the baud rate to 115200
  - Initialize the external PSRAM (if available) and allocate memory space for the image
  - Initialize the LCD display, including setting it to horizontal display mode, clearing the screen to white, creating a new image cache, and setting the relevant parameters
  - Perform a series of graphic drawing operations, including drawing points, lines, rectangles, circles, numbers, strings, and more, and display the image on the LCD
  - Read the data from the QMI8658 sensor and display it on the LCD. First, use `Paint_Clear(WHITE)` to clear the image cache, then redraw the graphics and display the sensor data and the voltage value after ADC conversion

Result demonstration

- After power-on, the screen first displays white, red, green, and blue colors in intervals of 2 seconds to check for any light leaks and black spots. If the screen is too fast to see clearly, please press the RESET button to restart the process

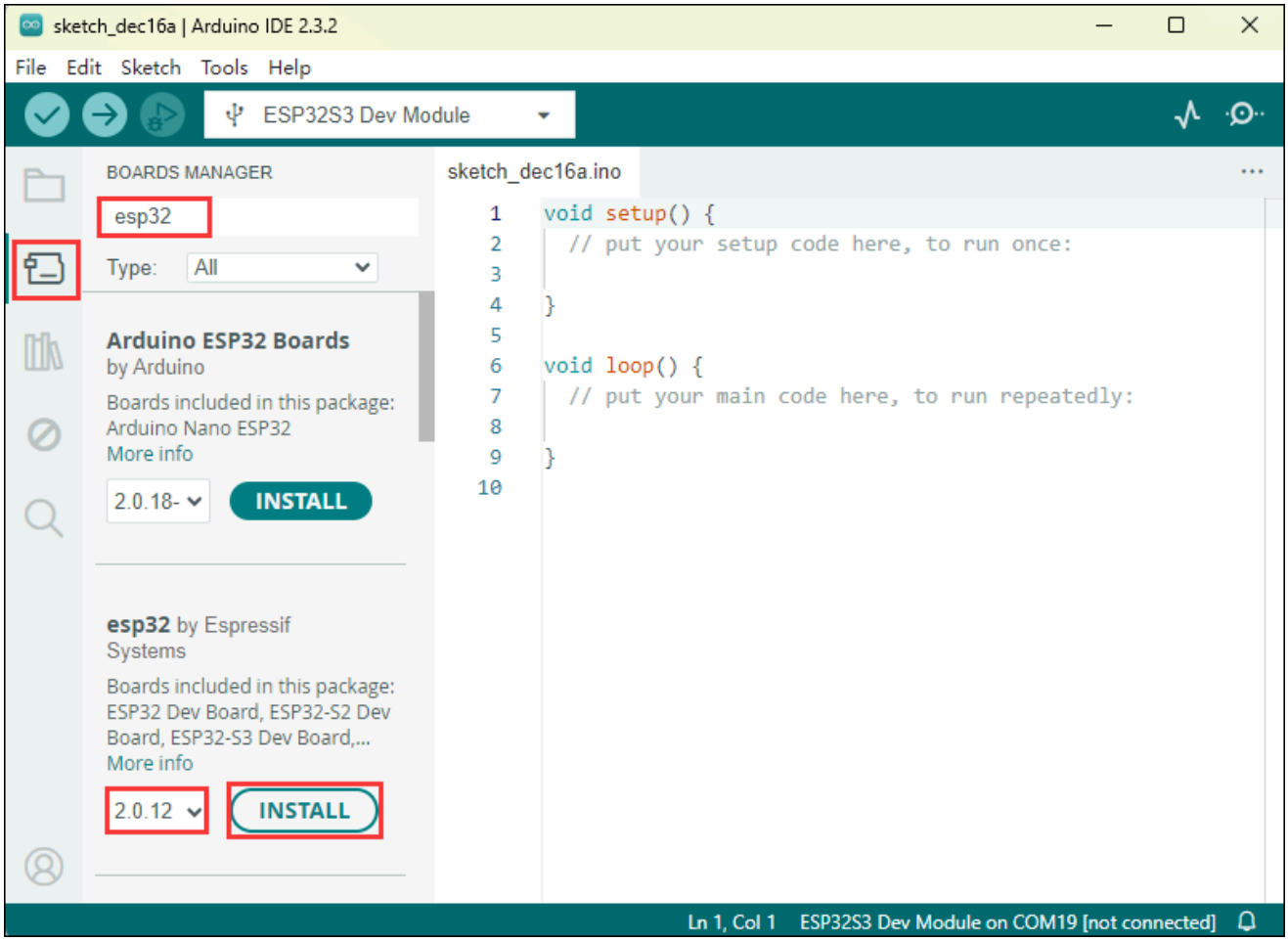
- Then enter the sensor test session, after the color display, the sensor data will be displayed on the screen, the ACC_X, ACC_Y and ACC_Z values will change with the angular deflection of the device, and the GYR_X, GYR_Y, and GYR_Z values will change with the change of the acceleration of the device
- Connect a 3.7V lithium battery at this moment. Under normal circumstances, the BAT (V) value will decrease
- Flash the code directly, the LCD screen is shown in the figure

(/wiki/File:300px-ESP32-S3-Touch-LCD-1.28-ArDemo-demo-01.png)

(/wiki/File:300px-ESP32-S3-Touch-LCD-1.28-ArDemo-demo-02.png)

(/wiki/File:300px-ESP32-S3-Touch-LCD-1.28-ArDemo-demo-03.png)

- If an error occurs, ensure that the ESP32 development board version is 2.0.12



(/wiki/File:ESP32-S3-LCD-1.28-demo-10.png)

LVGL_Arduino

[Collapse]

Demo description

- The demo is used to display LVGL benchmark, music, etc

Hardware connection

- Connect the development board to the computer

Code analysis

- **void my_disp_flush()**: Refreshes the image data from the LVGL to the TFT display. It receives LVGL display driver structure pointers, display area, and color data pointers as parameters
  - First calculates the width `w` and the height `h` of the area to be refreshed, this is done by obtaining the difference in coordinates of the input display area and

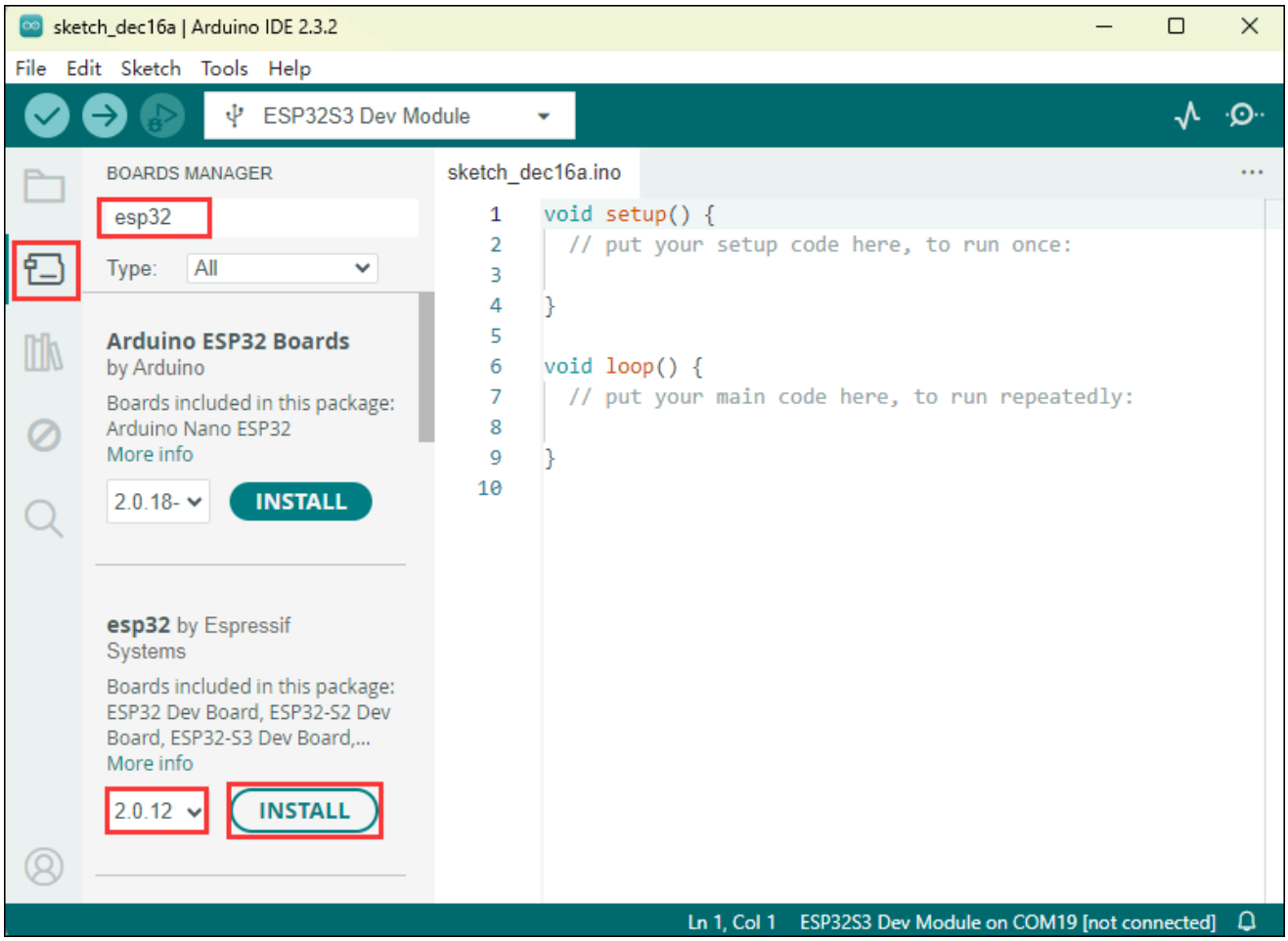
adding 1
    - Next call `tft.startWrite()` to start writing data to the TFT display, and then use `tft.setAddrWindow` to set the display area of the TFT display to the area specified by the incoming `area`
    - Then call `tft.pushColors` to push the color data (the color data pointing to the `color_p` is converted to the `uint16_t` type) to the TFT display for display, where the `w * h` parameter indicates the number of pixels to be pushed, `true` indicates that a data transfer completion operation is performed after the push operation is completed
    - Finally, call `lv_disp_flush_ready(disp_drv)` to notify LVGL of the completion of the refresh so that LVGL can proceed with subsequent operations
- **setup()**: Responsible for initializing serial communication, LVGL library, TFT display, and setting up LVGL display drivers, etc.
    - First initialize the serial port communication, set the baud rate to 115200, and print the version information of LVGL and Arduino
    - Initialize the LVGL library, initialize the TFT display, set it to horizontal display and flip, initialize the LVGL display buffer and display driver, register the `my_disp_flush` function as the LVGL refresh callback function
    - Create a LVGL image object and set the image source to `test1_240_240_4`
    - Create two timers, one to periodically increase the time ticks of LVGL, and the other to periodically execute the `example_increase_reboot` function
    - Finally print "Setup done" to indicate that the setup is complete

Result demonstration

(/wiki/File:200px-ESP32-S3-LCD-1.28-demo-03.png)

(/wiki/File:200px-ESP32-S3-LCD-1.28-demo-04.png)

- If an error occurs, ensure that the ESP32 development board version is 2.0.12



(/wiki/File:ESP32-S3-LCD-1.28-demo-10.png)

LVGL_Chinese_Font

[Collapse]

Demo description

- The demo is used to display 1000 commonly used Chinese fonts built into LVGL

Hardware connection

- Connect the development board to the computer

Code analysis

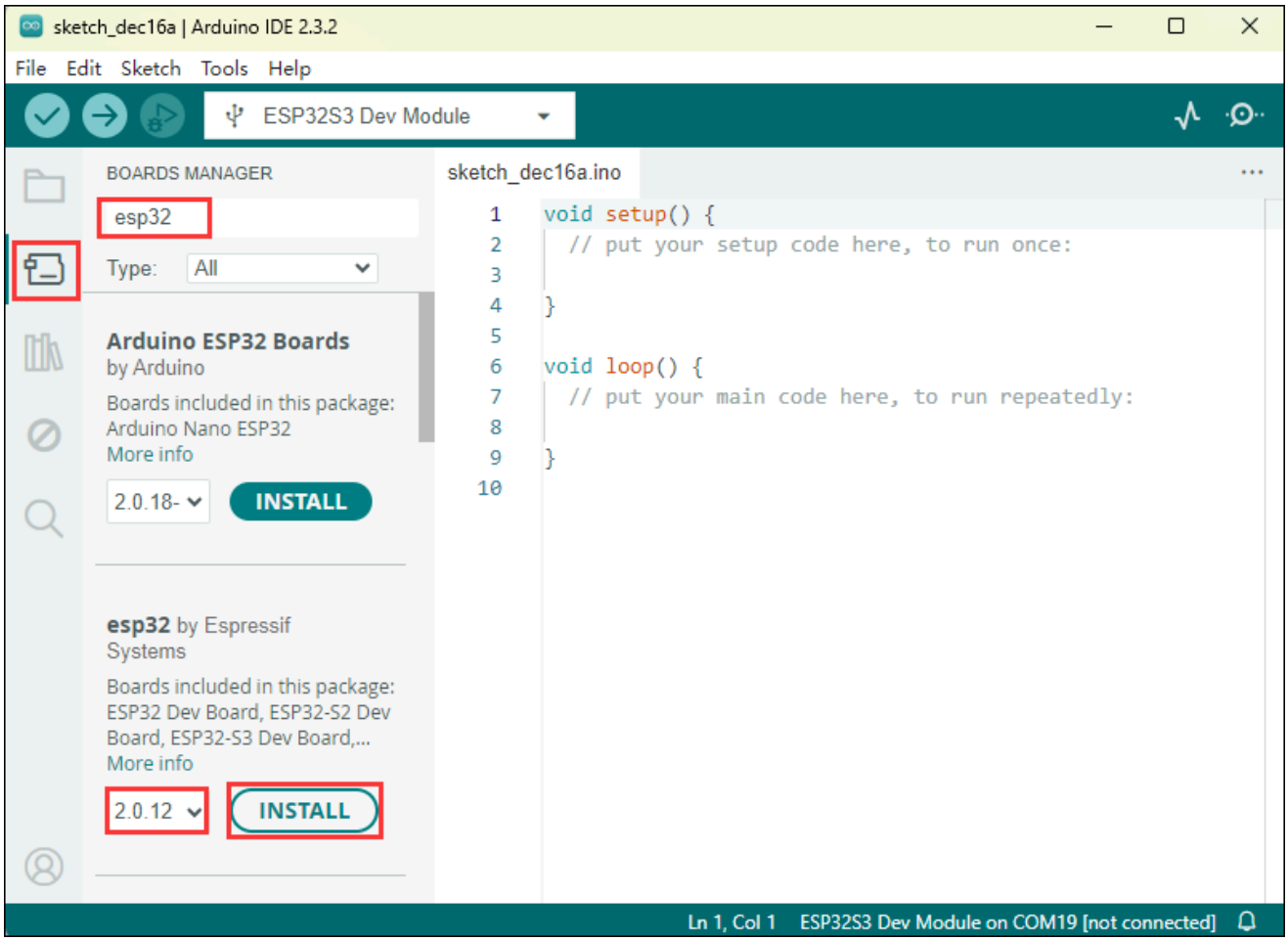
- **my_disp_flush()**
  - This function is the refresh callback function for the LVGL display drive
  - Calculate the width w and height h to be refreshed based on the input display area parameter area
  - Call the relevant functions of the TFT display: `tft.startWrite()`、`tft.setAddrWindow()` and `tft.pushColors()` to write the color data color_p from the LVGL to a specific area of the TFT display

- Finally call `lv_disp_flush_ready()` to notify LVGL that the refresh is complete

Result demonstration

(/wiki/File:400px-ESP32-S3-Touch-LCD-1.28-ArDemo-CH_fontl-01.png)

- If an error occurs, ensure that the ESP32 development board version is 2.0.12



(/wiki/File:ESP32-S3-LCD-1.28-demo-10.png)

LVGL_Chinese_7500_Char

[Collapse]

Demo description

- The demo is used to display 7500 Chinese fonts of LVGL, the font file is large, and the download of the firmware takes a long time

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup()**
  - First activate serial port communication, set the baud rate to 115200, and prepare for potential serial port debugging
  - Initialize LVGL (Light and Versatile Graphics Library), including setting the version information output, and registering the log print callback function (if log functionality is enabled)

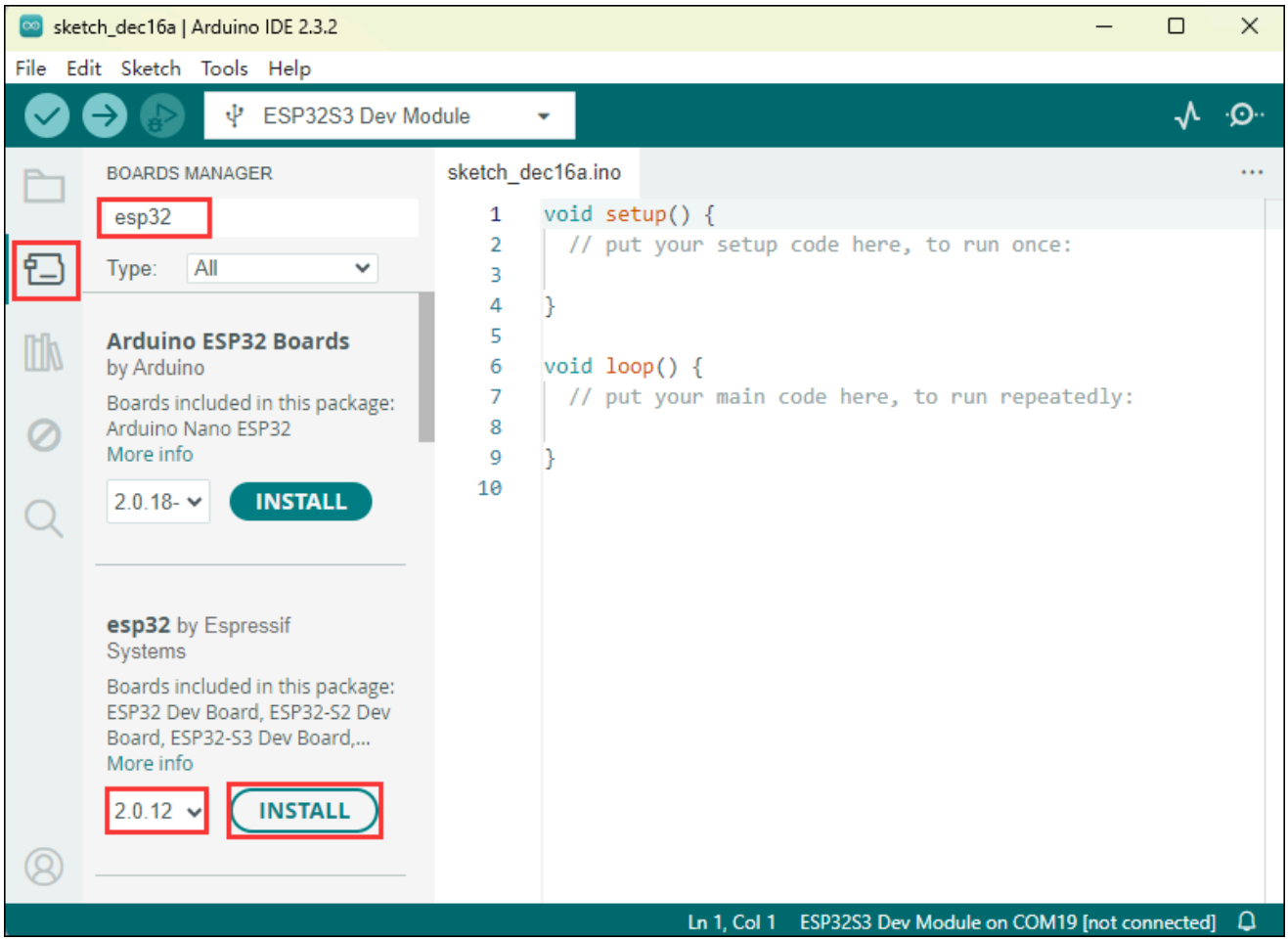
- Initialize the TFT display `tft` and `touch` sensor, including setting the rotation direction of the display and possible touch calibration data
- Initialize the LVGL display buffer `draw_buf` and display driver `disp_drv`, set the display resolution, refresh callback function, etc., and register the display driver with LVGL
- Initialize the input device driver `indev_drv`, set it to the pointer type and specify the touch read callback function, and then register the input device driver
- Create a label `label`, set the text font, text content (supporting text color recreation), and display it centrally on the screen
- You can optionally call LVGL's demo or demonstration functions to demonstrate different functional effects

- **loop()**
  - Call `lv_timer_handler()` function to let the LVGL graphics library handle its internal scheduled tasks and events
  - Use the `delay(5)` function to introduce a small delay to avoid excessive CPU usage by the demo

Result demonstration


(/wiki/File:400px-ESP32-S3-Touch-LCD-1.28-ArDemo-CH_7500-01.png)

- If an error occurs, ensure that the ESP32 development board version is 2.0.12



(/wiki/File:ESP32-S3-LCD-1.28-demo-10.png)

Working with MicroPython

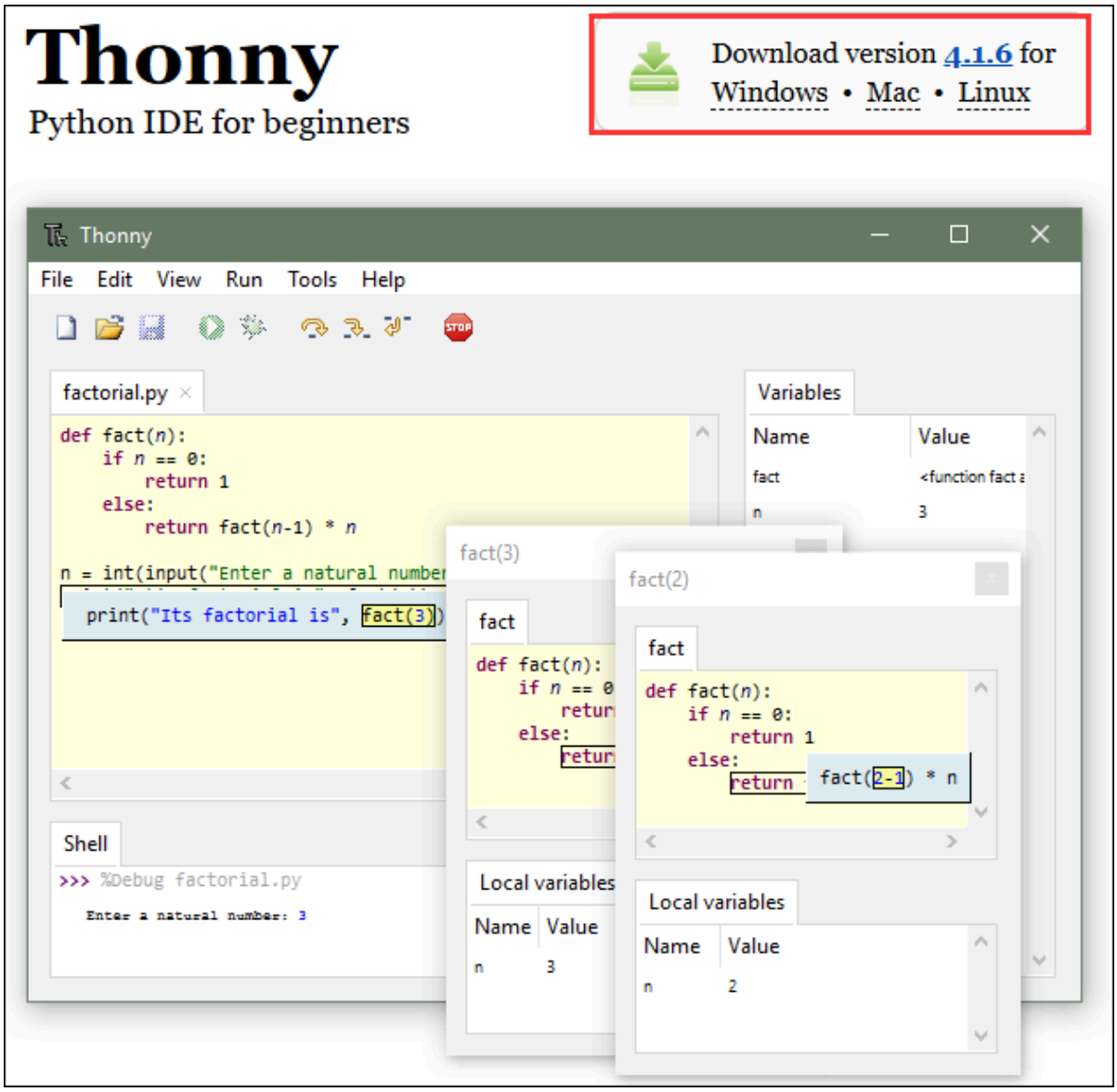
This section focuses on setting up the MicroPython development environment, primarily covering firmware flashing, installation, and the use of Thonny. For the installation part of Thonny, detailed explanations of the installation steps and usage instructions are provided, offering comprehensive and clear guidance for developers to build a MicroPython development environment.

(/wiki/File:MicroPython-flow-01.png)

Environment Setup

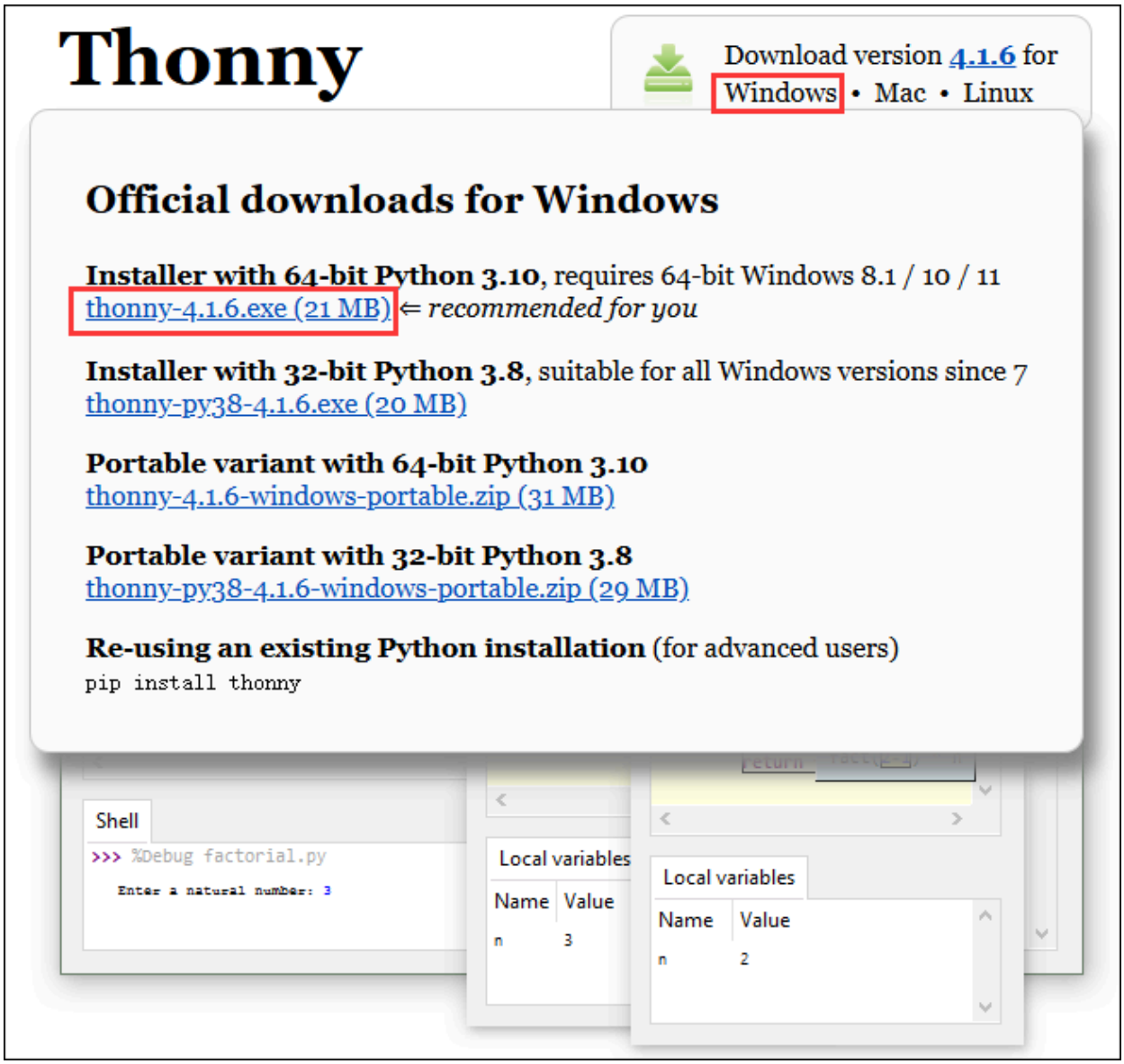
Download and install Thonny

- Open the download page of Thonny official website (https://thonny.org/) and select the corresponding system and system bit to download



(/wiki/File:ESP32-S3-Touch-LCD-Thonny_install-01.png)

- Select the appropriate system and version, for example, with a 64-bit Windows, the mouse needs to be moved to the Windows section to display the corresponding information, then click to Download and Install



(/wiki/File:ESP32-S3-Touch-LCD-Thonny_install-02.png)

- The rest can be installed by default

Flash the firmware

- Currently, the development board is developed using customized firmware, which is available in ESP32-S3-LCD-1.28 demo, the firmware is merged into a single file, note that the download address is the 0x00 location
- Firmware path:

```
...\ESP32-S3-LCD-1.28-Demo\Firmware\MicroPython-bin
```

- For a tutorial on flashing firmware, please refer to Flash Firmware Flashing and Erasing tutorial (https://www.waveshare.com/wiki/Flash_Firmware_Flashing_and_Erasing)
- The bin file in the factory-bin folder is a file for testing the onboard function, and there is no need to flash it here

> For MicroPython firmware creation, refer to link (https://github.com/russhughes/gc9a01_mpy), and refer to MicroPython firmware compilation (https://github.com/micropython/m

icropython/tree/master/ports/esp32) for more learning resources for MicroPython firmware compilation

Thonny Usage

If you are just getting started with ESP32 and Thonny, and you don't know how to use Thonny, please expand and take a look. Hope it can help you!

[Expand]

Demo

(/wiki/File:1000px-Demo-flow-02.png)

- ESP32-S3-LCD-1.28 demos

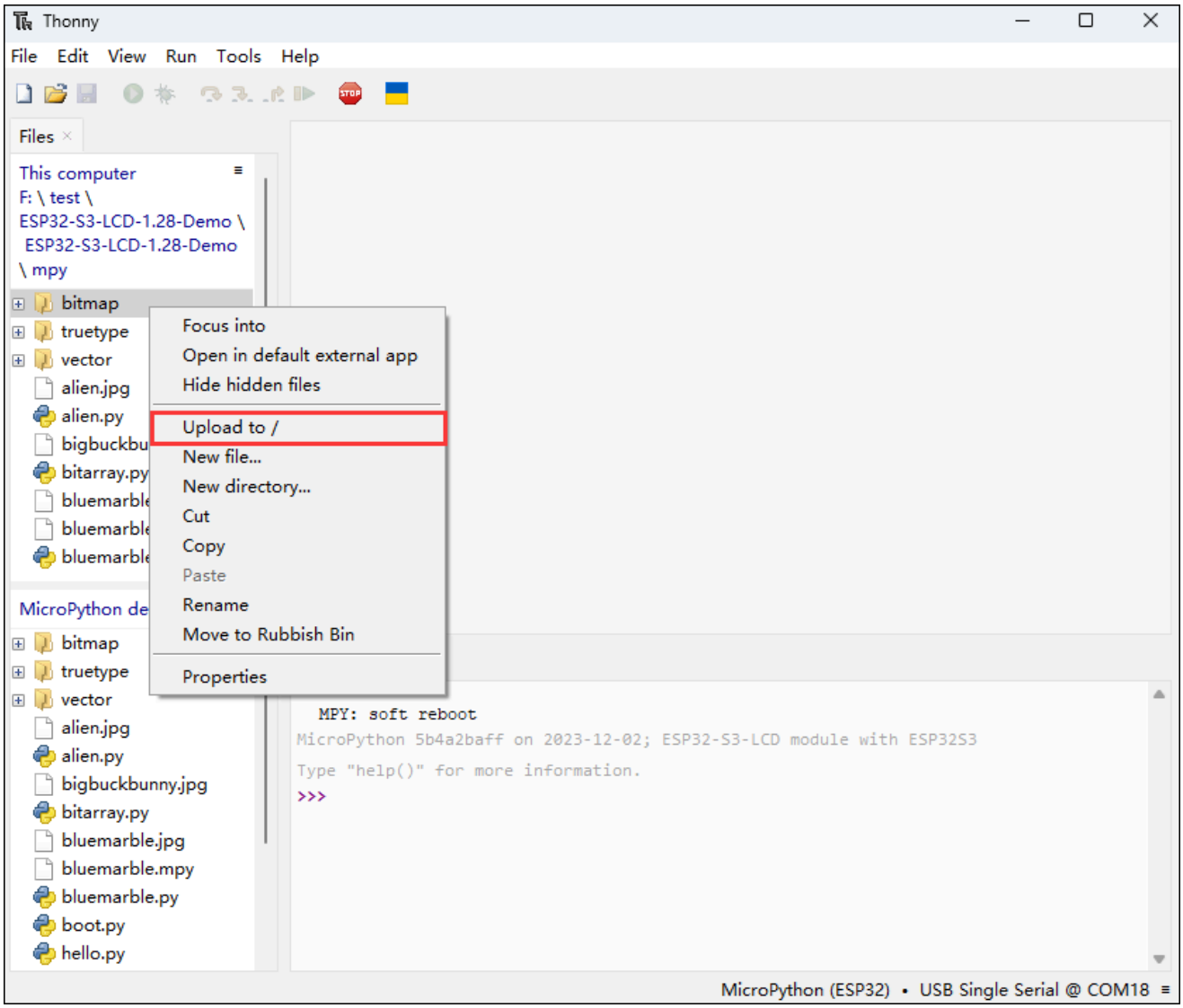
Demo	Basic Description
alien.py	Display the "alien.jpg" image randomly
bitarray.py	Create and display animation of the Pac-Man sprite in multiple random locations
hello.py	Randomly display different colors of "Hello!" text, and you can switch different rotation angles by looping
hershey.py	Loop through different greetings
jpg.py	Alternate between two JPEG images
noto_fonts.py	Displays the names of three different fonts
pbitmap.py	Display a precompiled bitmap image
rotation.py	Cycle through the text display at different rotation values
scroll.py	Implement smooth scrolling of characters on the display screen

Upload demo and special demos

[Collapse]

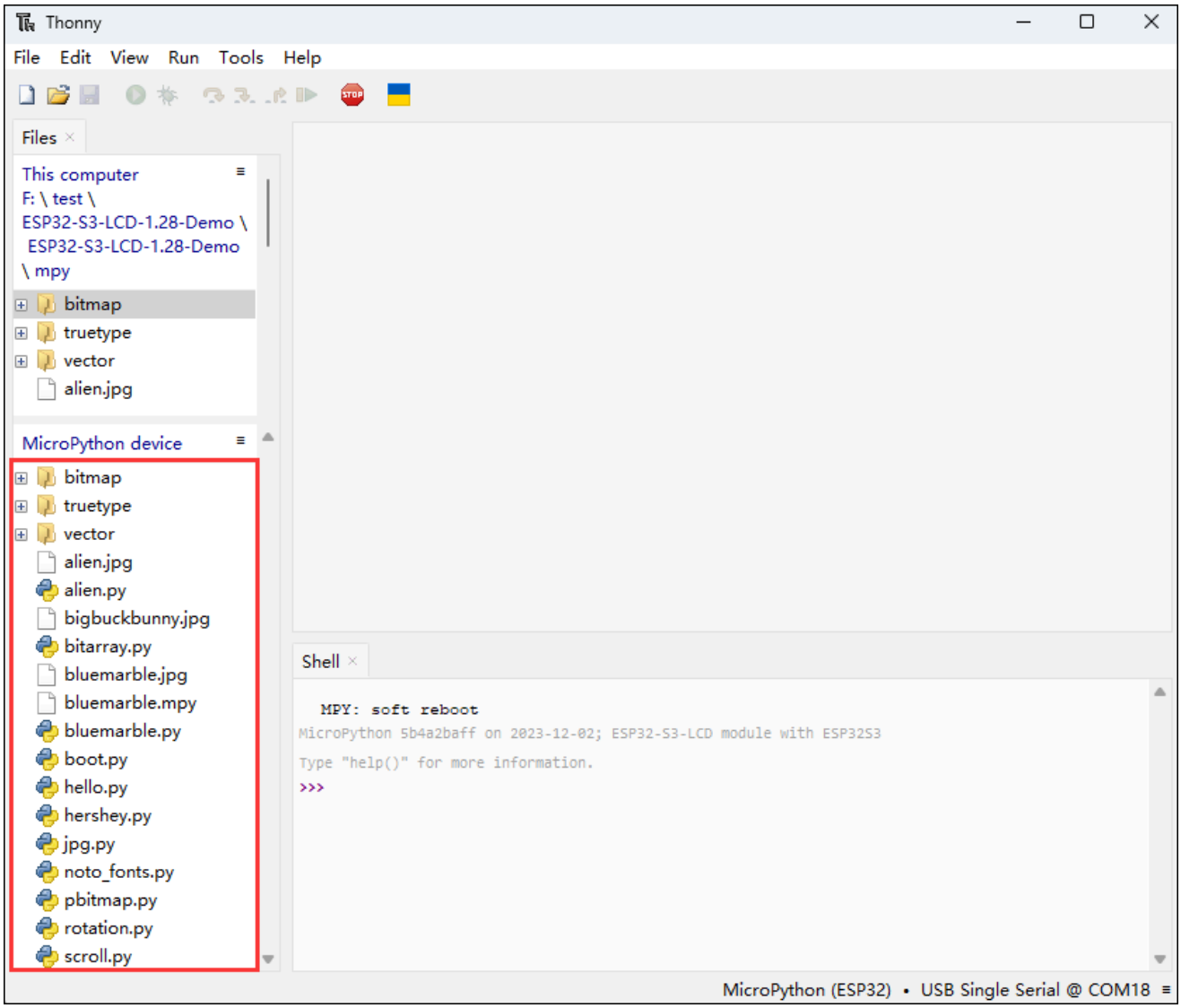
Upload demo

- Upload the local file to the development board, select the file, right-click with the mouse, find "Upload to/", and then download



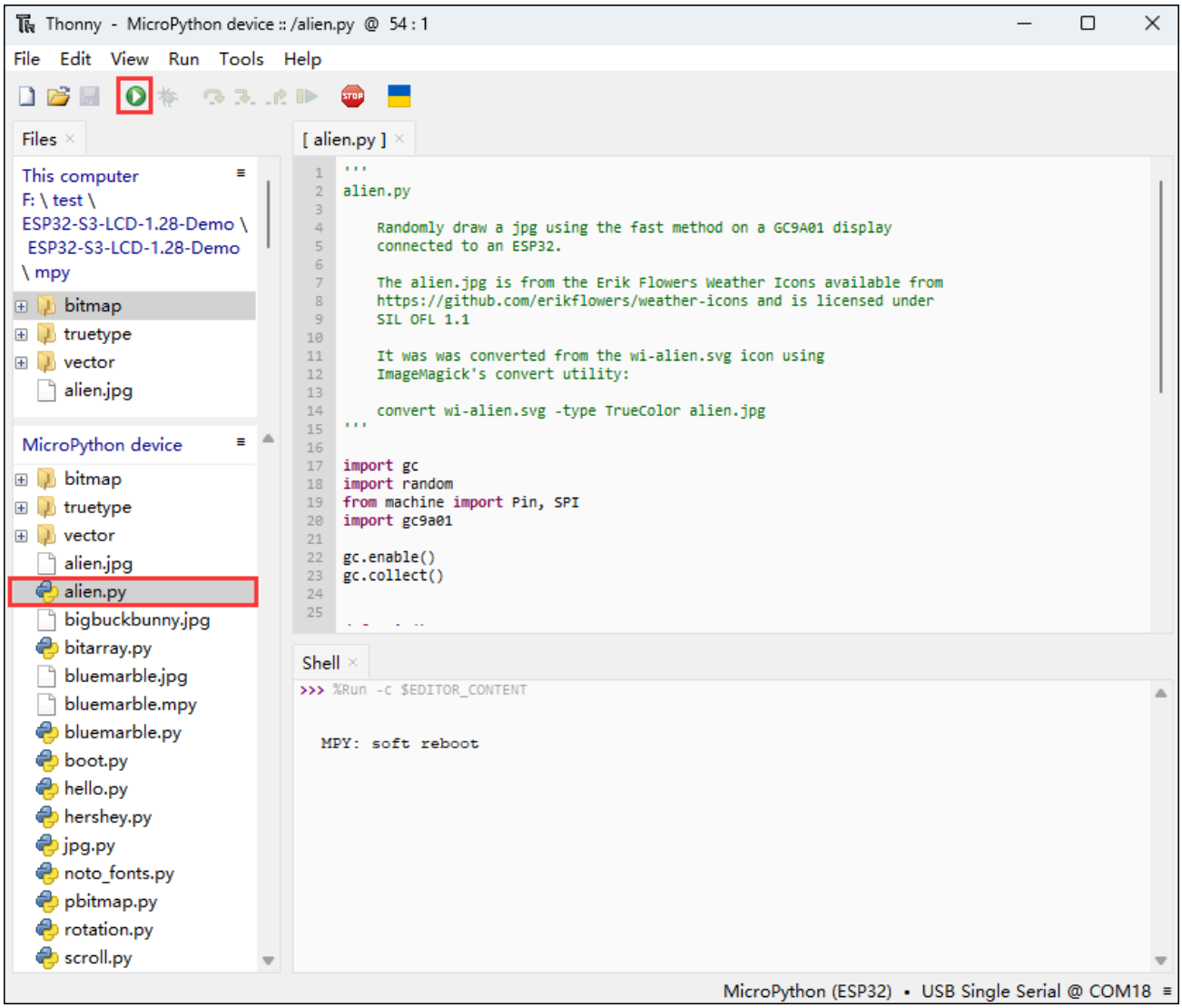
(/wiki/File:ESP32-S3-LCD-1.28-demo-05.png)

- The following is the interface where all downloads are completed, **the downloaded file must be consistent with the file in the red box**, otherwise it may fail to run



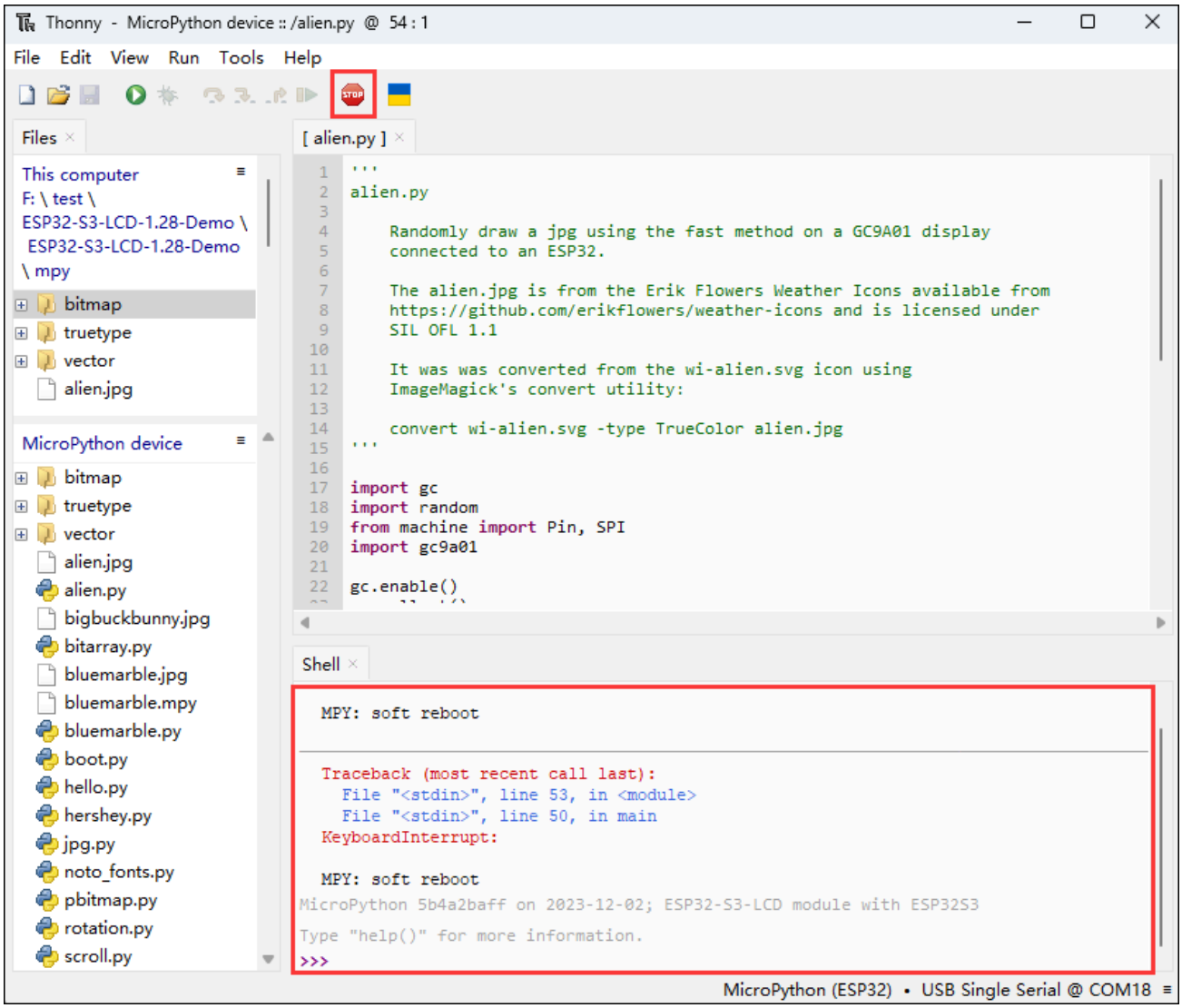
(/wiki/File:ESP32-S3-LCD-1.28-demo-06.png)

- Select the file with the .py suffix, and click the green button to flash and run (the operation figure is as follows)



(/wiki/File:ESP32-S3-LCD-1.28-demo-07.png)

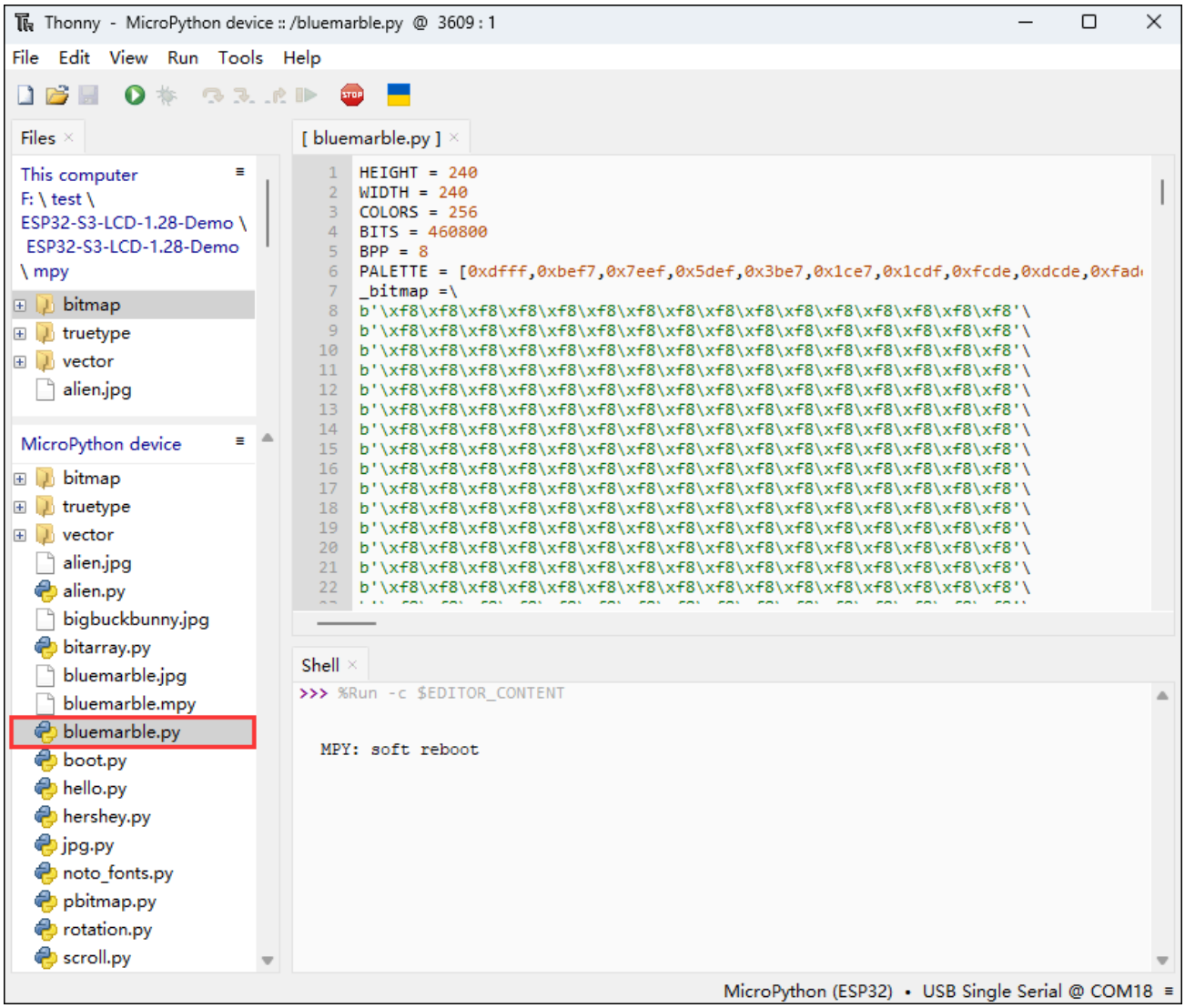
- When running another file, first click the red stop button, so that the other file can run normally



(/wiki/File:ESP32-S3-LCD-1.28-demo-08.png)

Upload special demos

- In the given demo, the boot.py is an automatically generated blank file, and the bluenarble.py stores images, **nothing will happen if you run the file**



(/wiki/File:ESP32-S3-LCD-1.28-demo-09.png)

alien.py

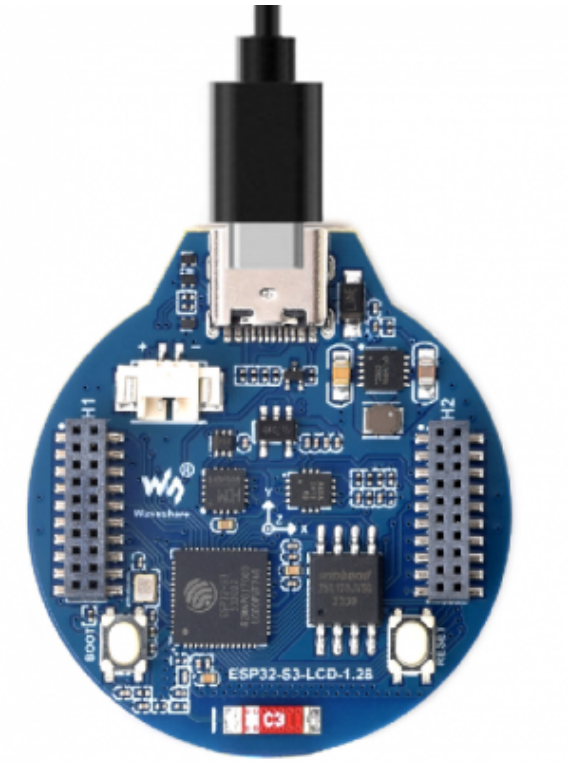
[Collapse]

Demo description

- This demo is used to randomly display the "alien.jpg" image on a specific display

Hardware connection

- Connect the development board to the computer



(/wiki/File:300px-ESP32-S3-LCD-1.28-demo-01.png)

Code analysis

- **GC9A01()**: The importance of the GC9A01 constructor is to establish the hardware connection, set the initial parameters, and create an operable display object that lays the foundation for subsequent display operations
  - `spi_interface` : Establish SPI communication with the display screen
  - `width` and `height` : Set the display resolution
  - `reset_pin` : Connect various control pins of the display screen
  - `rotation` : Set the initial rotation angle

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-alien-01.png)

bitarray.py

[Collapse]

Demo description

- The demo creates and displays multiple animations of the Pac-Man sprite in random locations on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **pacman class move method**: Responsible for updating the status of the Pac-Man sprite to move and animate it on the screen
  - First increase the current number of steps of the sprite, and then make sure that the number of steps loops within the specified range of `SPRITE_STEPS` by taking the conversion, corresponding to different animation frames or states of the sprite
  - Then, move the sprite's position in the horizontal direction. When the sprite's horizontal position reaches a specific value of 302, reset the step count. This is done to trigger a specific animation state or behavior. Finally, the conversion operation is used to ensure that the horizontal position of the sprite cycles within a certain range to avoid exceeding the display area
- **tft.map_bitarray_to_rgb565**
  - `tft.map_bitarray_to_rgb565` function selects bitmap data from the `SPRITE_BITMAPS` based on the current steps of the sprite, converts it into an RGB565 format buffer

named `sprite`. At the same time, the width of the sprite, as well as the foreground and background colors, were specified

- **tft.blit_buffer**
  - `tft.blit_buffer` function draws the converted buffer to a specific location on the display screen, which is determined by the current coordinates of the sprite and specifies its width and height

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-bitarray-02.png)

hello.py

[Collapse]

Demo description

- The demo randomly displays different colors of "Hello!" text on a specific display, and different rotation angles can be switched by cycling

Hardware connection

- Connect the development board to the computer

Code analysis

- **tft.text**: Draw the text "Hello!" on the display
  - Using the specified font, the position of the text is determined within the range of the display by random numbers, and the foreground and background colors are

determined by the randomly generated RGB565 color values

- **while True** : Implement the dynamic effect of random display of "Hello!" text at different rotation angles
  - By using an infinite loop and iterating through four rotation angles, set the display rotation angle, clear the screen, calculate the text display range, and cyclically call the `tft.text` function to display text at random positions in random colors

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-hello-03.png)

hershey.py

[Collapse]

Demo description

- The demo loops through different greetings on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The loop section of the function that cycles through different fonts, colors, and greetings on the display
  - Move the display position line by line, get the next color, font, and greeting, clear the previous line and draw new content, reset the line position when it exceeds a

certain range, add a delay to control the display speed

- **cycle** : Create an object that can be iterated over in a loop
  - Accept parameters. If iterable, loop directly. If a single element, convert it into an iterable list and loop, facilitating the convenient looping of color, font, and greeting lists

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-hershey-04.png)

jpg.py

[Collapse]

Demo description

- This demo alternates displaying two JPEG images on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The loop section of the function that cycles through different fonts, colors, and greetings on the display
  - Iterate through the list of image filenames, call the `tft.jpg` function to display the image in a slower way at a specific location, and wait 5 seconds

- **tft.jpg** : Display the specified JPEG picture on the display
  - Read, decode picture files and draw to display screen, determine display position and mode according to incoming parameters

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-jpg-05.png)

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-pbitmap-07.png)

noto_fonts.py

[Collapse]

Demo description

- The demo displays the names of three different fonts (NotoSans, NotoSerif, NotoSansMono) on a specific display, and these font names are shown in the center on separate lines

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The main entrance of the demo, which initializes the display and displays three font names
  - Create a display object and initialize it, fill the background in black, and call the center function to display the three font names in turn
- **center** : Display a given string in the center of the display in the specified font
  - Get the screen width, calculate the width of the string, if it is less than the screen width, calculate the coordinates for horizontal centering, otherwise for left alignment. Finally, display the string at the specified location

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-noto-fonts-06.png)

pbitmap.py

[Collapse]

Demo description

- This demo displays a pre-compiled bitmap image on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The main entrance of the demo, which initializes the display and displays the bitmap
  - Create a display object, fill it with a black background after initialization, and call `tft.pbitmap` to display the image in the pre-compiled bitmap module

tft.pbitmap**: Display pre-compiled bitmap on the screen**

- - Read the bitmap data and draw it to the display, and determine the display position based on the incoming parameters

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-pbitmap-07.png)

rotation.py

[Collapse]

Demo description

- The demo loops through the text display at different rotation values on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The main entrance of the demo, which initializes the display and loops through the text display at different rotation values
  - Create a display object and initialize it, set different rotation values in turn in the loop, clear the screen and display the text with the rotation value information and wait for observation
- **tft.rotation**: Set the display rotation angle
  - Adjust the display orientation according to the incoming parameters

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-rotation-08.png)

scroll.py

[Collapse]

Demo description

- This demo implements a smooth scrolling display of characters on a specific display

Hardware connection

- Connect the development board to the computer

Code analysis

- **main**: The main entrance of the demo, which initializes the display, sets the color, and scrolls through the characters
  - Create a display object, set up a color generator to get the foreground color, set the scroll area after initializing the display, clear the top row in the loop, display characters on the new line and update the color and character values, set the scroll address for scrolling and control the speed
- **cycle** : Create an object that can be iterated over in a loop
  - Accept parameters. If iterable, loop directly. If a single element, convert it into an iterable list and loop

Result demonstration

- LCD screen display

(/wiki/File:300px-ESP32-S3-Touch_1.28-MP-Thonny_demo-scroll-09.png)

Flash Firmware Flashing and Erasing

- **The current demo provides test firmware, which can be used to test whether the onboard device functions properly by directly flashing the test firmware**
- **bin file path:**

```
...\ESP32-S3-LCD-1.28-Demo\Firmware\factory-bin
```

> Flash firmware flashing and erasing (https://www.waveshare.com/wiki/Flash_Firmware_Flashing_and_Erasing) for reference

Resources

Schematic diagram

- ESP32-S3-LCD-1.28 Schematic diagram (https://files.waveshare.com/wiki/ESP32-S3-LCD-1.28/Esp32-s3-lcd-.128-sch.pdf)

Project diagrams

- ESP32-S3-LCD-1.28 3D diagram (https://files.waveshare.com/wiki/ESP32-S3-LCD-1.28/ESP32-S3-LCD-1.28.zip)

Demo

- ESP32-S3-LCD-1.28 Demo (https://files.waveshare.com/wiki/ESP32-S3-LCD-1.28/ESP32-S3-LCD-1.28-Demo.zip)

Application Examples

- Arduino Temperature Gauge (https://www.youtube.com/watch?v=A00CvNi1rzQ)
- Inclinometer (https://www.youtube.com/watch?v=GosqWcScwC0)
- CHEAP DIY BOOST GAUGE (https://www.youtube.com/watch?v=cZTx7T9uwA4)

Datasheets

ESP32-S3

- ESP32-S3 Technical Reference Manual (https://files.waveshare.com/wiki/common/Esp32-s3_technical_reference_manual_en.pdf)
- ESP32-S3 Series Datasheet (https://files.waveshare.com/wiki/common/Esp32-s3_datasheet_en.pdf)
- ESP32-S3 Technical Documents (https://www.espressif.com/en/support/documents/technical-documents?keys=&field_type_tid%5B%5D=842)

Other components

- GC9A01A Datasheet (https://files.waveshare.com/wiki/common/GC9A01A.pdf)

- QMI8658A Datasheet (https://files.waveshare.com/wiki/common/QMI8658A_Datasheet_Rev_A.pdf)

Software tools

Arduino

- Arduino IDE Official download link (https://www.arduino.cc/en/software/)

Thonny

- Thonny official download link (https://thonny.org/)

Firmware flashing tool

- Flash_download_tool (https://dl.espressif.com/public/flash_download_tool.zip)

Other resource link

- ESP32-Arduino official documentation (https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html)
- MicroPython official documentation (https://docs.micropython.org/en/latest/)
- LVGL official documentation (https://docs.lvgl.io/master/intro/introduction/index.html)

FAQ

Question: After the module downloads the demo and re-downloads it, why sometimes it can't connect to the serial port or the flashing fails?

Answer:

- Click the Reset button for more than 1 second, wait for the PC to re-recognize the device and then download again
- Long press the BOOT button, press RESET at the same time, then release RESET, then release the BOOT button, at this time the module can enter the download mode, which can solve most of the problems that can not be downloaded.

Question: Why does the module keep resetting and flicker when viewed the recognition status from the device manager?

Answer:

It may be due to Flash blank and the USB port is not stable, you can long-press the BOOT button, press RESET at the same time, and then release RESET, and then release the BOOT button, at this time the module can enter the download mode to flash the firmware (demo) to solve the situation.

Question: How to deal with the first compilation of the program being extremely slow?

Answer:

- It's normal for the first compilation to be slow, just be patient

Question: What should I do if I can't find the AppData folder?

Answer:

- Some AppData folders are hidden by default and can be set to show.
- English system: Explorer->View->Check "Hidden items"
- Chinese system: File Explorer -> View -> Display -> Check "Hidden Items"

Question: How do I check the COM port I use?

Answer:

- Windows system:

①View through Device Manager: Press the Windows + R keys to open the "Run" dialog box; input devmgmt.msc and press Enter to open the Device Manager; expand the "Ports (COM and LPT)" section, where all COM ports and their current statuses will be listed.
②Use the command prompt to view: Open the Command Prompt (CMD), enter the "mode" command, which will display status information for all COM ports.
③Check hardware connections: If you have already connected external devices to the COM port, the device usually occupies a port number, which can be determined by checking the connected hardware.

- Linux system:

①Use the dmesg command to view: Open the terminal.
①Use the ls command to view: Enter ls /dev/ttyS* or ls /dev/ttyUSB* to list all serial port devices.

③Use the setserial command to view: Enter setserial -g /dev/ttyS* to view the configuration information of all serial port devices.

Question: Why does the program flashing fail when using a MAC device?

Answer:

- Install MAC Driver (https://files.waveshare.com/wiki/common/CH34XSER_MAC.7z) and flash again.

Question: What is the screen driver?

Answer:

GC9A01A

Question: What battery does ESP32-S3-LCD-1.28 use to power it?

Answer:

MX1.25 2P connector, for 3.7V battery, supports charging and discharging

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help

Submit Now (https://service.waveshare.com/)

you to resolve the issue.
Working Time: 9 AM - 6 PM GMT+8
(Monday to Friday)

Retrieved from "https://www.waveshare.com/w/index.php?title=ESP32-S3-LCD-1.28&oldid=107025 (https://www.waveshare.com/w/index.php?title=ESP32-S3-LCD-1.28&oldid=107025)"